Unit 8.2 Advanced Programming – Revision Sheet

Cambridge IGCSE Computer Science · Procedures and functions · File handling · Maintainable programs · Databases and SQL · Boolean logic. Everything on the specification for this test, with pseudocode in the Cambridge style (← for assignment, // for comments).

1 Procedures and functions

Key words

- **Subroutine (subprogram):** a named, self-contained block of code that does one task and can be called from anywhere in the program.
- **Procedure:** a subroutine that performs a task but does **not return a value**. It is run with `CALL`.
- **Function:** a subroutine that performs a task and **returns a single value**. It is used inside an expression (assignment, `OUTPUT`, `IF`) – **never** with `CALL`.
- **Parameter:** a variable named in the subroutine header that receives a value when the subroutine is called. Parameters are **local** to the subroutine.
- **Argument:** the actual value (or variable) passed to a parameter in the call.
- **Calling:** running a subroutine by its name; the program jumps to it, runs it, then returns to the line after the call.
- **Return value:** the value a function sends back with `RETURN`.
- In the exam, procedures and functions have **up to three parameters** (or none).

	Procedure	Function
Returns a value?	No	Yes – exactly one
Header	`PROCEDURE Name(P : TYPE)`	`FUNCTION Name(P : TYPE) RETURNS TYPE`
Ends with	`ENDPROCEDURE`	`ENDFUNCTION`
Called by	`CALL Name(Arg)`	In an expression: `X ← Name(Arg)`

Syntax

```
PROCEDURE ShowMenu                      // no parameters
    OUTPUT "1. New game"
    OUTPUT "2. Quit"
ENDPROCEDURE
CALL ShowMenu

PROCEDURE ShowStars(Number : INTEGER) // one parameter
    DECLARE Line : STRING
    Line ← ""
    FOR Count ← 1 TO Number
        Line ← Line & "*"
    NEXT Count
    OUTPUT Line
ENDPROCEDURE
CALL ShowStars(5)                       // outputs *****
```

```
FUNCTION Square(Number : INTEGER) RETURNS INTEGER
    RETURN Number * Number
ENDFUNCTION

Result ← Square(5)            // Result = 25
OUTPUT Square(4)              // outputs 16
IF Square(X) > 100 THEN OUTPUT "Big" ENDIF
```

```
FUNCTION Max(A : INTEGER, B : INTEGER) RETURNS INTEGER
    IF A > B
      THEN RETURN A
      ELSE RETURN B
    ENDIF
ENDFUNCTION

// three parameters:
FUNCTION Mean(A : REAL, B : REAL, C : REAL) RETURNS REAL
    RETURN (A + B + C) / 3
ENDFUNCTION

FUNCTION IsEven(Number : INTEGER) RETURNS BOOLEAN
    RETURN MOD(Number, 2) = 0
ENDFUNCTION
```

- Multiple parameters are separated by **commas**, each with its own data type.
- Arguments must match the parameters in **number, order and data type**: the first argument goes to the first parameter, and so on.
- A function's return value must be **used** (stored, output or tested) – otherwise it is lost.

Why use procedures and functions?

- Code is written **once and reused** → shorter programs, less repetition.
- Each subroutine can be **tested and debugged on its own**.
- A change or fix is made in **one place** and applies to every call.
- Programs are **easier to read and maintain** (each part has a clear name and job).
- Tasks can be **split between programmers**; subroutines can be reused in other programs.
- Supports **decomposition** – a big problem is broken into smaller sub-problems.

Local and global variables

- **Local variable:** declared **inside** a subroutine. Can only be used inside that subroutine and only exists while it runs.
- **Global variable:** declared in the **main program**. Can be used anywhere in the program, for the whole time it runs.
- **Scope:** the part of the program where a variable can be accessed.
- If a local and a global variable share a name, the **local** one is used inside the subroutine.

```
DECLARE Total : INTEGER          // global
Total ← 0

PROCEDURE AddScore(Score : INTEGER)
    DECLARE Bonus : INTEGER      // local
    Bonus ← 5
    Total ← Total + Score + Bonus
ENDPROCEDURE

CALL AddScore(10)
OUTPUT Total                 // 15
OUTPUT Bonus                 // ERROR - Bonus is local:
                             // it only exists in AddScore
```

Why local variables are preferred: they can't be changed by mistake elsewhere; the same name can be reused in different

subroutines; memory is freed when the subroutine ends; subroutines are self-contained, so easier to test and reuse.

Drawback of globals: any part of the program can change them, so errors are hard to trace and subroutines that rely on them are harder to reuse. They suit values needed throughout the program (e.g. a running score) – passing a parameter is usually better.

Library routines

Library routine: a ready-made, pre-tested subroutine supplied with the language that any program can call.
Benefits: saves time (no need to write or test it), already tested so reliable, written by experts so efficient, reusable.

Routine – what it does	Examples
`MOD(a, b)` remainder after dividing a by b	`MOD(17, 5) = 2` `MOD(20, 4) = 0`
`DIV(a, b)` whole-number part of a ÷ b (integer division)	`DIV(17, 5) = 3` `DIV(20, 4) = 5`

`ROUND(value, places)`
rounds to the given number of decimal places
	`ROUND(6.97354, 2) = 6.97`
	`ROUND(7.856, 1) = 7.9`
	`ROUND(7.5, 0) = 8`

`RANDOM()` random real number between 0 and 1 inclusive	`e.g. 0.384…`

```
IF MOD(Number, 2) = 0 THEN ...  // even (1 = odd)
IF MOD(X, Y) = 0 THEN ...       // multiple of Y
LastDigit ← MOD(472, 10)        // 2
Rest      ← DIV(472, 10)        // 47
Hours     ← DIV(135, 60)        // 2  } 135 min =
Minutes   ← MOD(135, 60)        // 15 } 2 h 15 min
Dice ← ROUND(RANDOM() * 5, 0) + 1    // 1 to 6
```

Dice: `RANDOM() * 5` gives 0 to 5, `ROUND(…, 0)` makes it a whole number, `+ 1` shifts it to 1 to 6. In general for 1 to N: `ROUND(RANDOM() * (N - 1), 0) + 1`.

2 File handling

Why store data in a file?

- Variables are held in **RAM** and are **lost** when the program ends. A file on secondary storage keeps data **permanently**.
- Data can be **loaded again next time** instead of being typed in again.
- Data can be **shared** with other programs or users, and large amounts can be stored.
- Examples: high scores, user settings, a list of customers, a log of results.

Commands

Task	Pseudocode
Open to read	`OPENFILE "Data.txt" FOR READ`
Open to write	`OPENFILE "Data.txt" FOR WRITE`
Read a line / item into a variable	`READFILE "Data.txt", Line`
Write a line / item	`WRITEFILE "Data.txt", Line`
Close the file	`CLOSEFILE "Data.txt"`
End of file reached?	`EOF("Data.txt")`

Rules

- Order: **open → read or write → close**.
- Open in the right **mode**: a file opened `FOR READ` can't be written to, and vice versa.
- `FOR WRITE` **overwrites** an existing file (it starts empty) and creates it if it doesn't exist.
- Each `READFILE` reads the **next** line; each `WRITEFILE` writes a **new line**.
- **Always close** the file: makes sure all data is saved, frees the file for other programs, releases resources.
- A **single item** is one value on its own line (e.g. a score); a **line of text** may hold several words or values read in one go.
- Use `WHILE NOT EOF(...)` when the number of lines isn't known; a `FOR` loop when it is.
- To **add** to existing data: read it into an array, add the new item, then write it all back `FOR WRITE`.

Single items

```
// FOR WRITE replaces the old score
OPENFILE "HighScore.txt" FOR WRITE
WRITEFILE "HighScore.txt", HighScore
CLOSEFILE "HighScore.txt"

OPENFILE "HighScore.txt" FOR READ
READFILE "HighScore.txt", HighScore
CLOSEFILE "HighScore.txt"
```

A line of text

```
Line ← FirstName & " " & Surname
OPENFILE "Names.txt" FOR WRITE
WRITEFILE "Names.txt", Line
CLOSEFILE "Names.txt"
```

Read and output every line

```
OPENFILE "Names.txt" FOR READ
WHILE NOT EOF("Names.txt") DO
    READFILE "Names.txt", Name
    OUTPUT Name
ENDWHILE
CLOSEFILE "Names.txt"
```

Write an array to a file

```
OPENFILE "Names.txt" FOR WRITE
FOR Index ← 1 TO 3
    WRITEFILE "Names.txt", Names[Index]
NEXT Index
CLOSEFILE "Names.txt"
```

Count lines · total numbers

```
Count ← 0
Total ← 0
OPENFILE "Scores.txt" FOR READ
WHILE NOT EOF("Scores.txt") DO
    READFILE "Scores.txt", Score
    Count ← Count + 1
    Total ← Total + Score
ENDWHILE
CLOSEFILE "Scores.txt"
OUTPUT Count, " scores, total ", Total
```

Search a file

```
INPUT Target
Found ← FALSE
OPENFILE "Names.txt" FOR READ
WHILE NOT EOF("Names.txt") AND Found = FALSE DO
    READFILE "Names.txt", Name
    IF Name = Target THEN Found ← TRUE ENDIF
ENDWHILE
CLOSEFILE "Names.txt"
IF Found
  THEN OUTPUT "Found"
  ELSE OUTPUT "Not found"
ENDIF
```

Copy one file into another

```
OPENFILE "Old.txt" FOR READ
OPENFILE "New.txt" FOR WRITE
WHILE NOT EOF("Old.txt") DO
    READFILE "Old.txt", Line
    WRITEFILE "New.txt", Line
ENDWHILE
CLOSEFILE "Old.txt"
CLOSEFILE "New.txt"
```

Common mistakes: forgetting to close the file · wrong mode (writing to a file opened FOR READ) · forgetting that FOR WRITE wipes the file · missing the file name or comma in READFILE/WRITEFILE · reading inside the wrong loop.

3 Maintainable programs

Maintainable program: one that other programmers (or you, later) can easily **read, understand, debug, update and extend**. Programs are changed and fixed over time, often by different people, so clear code makes this quicker, cheaper and less likely to add errors.

How to make a program maintainable

1. **Meaningful identifiers** for **variables, constants, arrays, procedures and functions** – names that say what they store or do: `TotalScore` not `X`; `TestMarks` (array); `CalculateVAT` (function); `MaxPlayers` (constant).
2. **Comments** (the commenting feature of the language, `//` in pseudocode) that are **relevant and appropriate**: explain the purpose of the program and of each subroutine, and any complex or non-obvious code – not every simple line.
3. **Procedures and functions** – code split into named tasks that can be understood, tested and changed one at a time, without repetition.
4. **Indentation and white space** – show which lines are inside loops, IFs and subroutines.
5. **Constants** instead of fixed values – the name explains the value and it is changed in one place.

Identifier rules: start with a letter; no spaces (use CamelCase `TotalScore` or snake_case `total_score`); can't be a keyword such as `IF` or `OUTPUT`.

Too many comments clutter the code and can go out of date; comments that state the obvious add nothing.

Hard to maintain

```
x ← 0
FOR i ← 1 TO 30
INPUT y
x ← x + y
NEXT i
z ← x / 30
OUTPUT z
```

Easy to maintain

```
// Calculates the average mark for a class
CONSTANT ClassSize ← 30
DECLARE TotalMark : INTEGER
DECLARE Mark : INTEGER
DECLARE AverageMark : REAL
TotalMark ← 0
FOR Student ← 1 TO ClassSize
    INPUT Mark
    TotalMark ← TotalMark + Mark     // running total
NEXT Student
AverageMark ← TotalMark / ClassSize
OUTPUT AverageMark
```

Exam tip: for "explain how this program has been made maintainable", name the feature **and** quote an example from the code, e.g. "the constant `ClassSize` means the class size only has to be changed in one place".

4 Databases and SQL

Key words

- **Database:** a structured, persistent collection of related data that can be searched, sorted and updated.
- **Single-table database:** all the data is held in one table.
- **Table:** data about one type of thing, in rows and columns.
- **Field:** one column – one item of data stored about every record (e.g. `Surname`).
- **Record:** one row – all the data about one item or person.
- **Primary key:** a field whose value is **unique** for every record, used to identify each record. Choose a field that is always unique and never empty, e.g. `StudentID` – not a name, surname or date of birth (these can repeat).
- **Validation:** automatic checking that data entered is **reasonable** and follows rules. It **can't** check data is correct (a mistyped but realistic age passes).
- **Verification** (not validation): checking data was copied or entered accurately, e.g. double entry or a visual check.

Data types

Type	Holds	Use for
Text / alphanumeric	letters, digits, symbols	names, postcodes, **phone numbers**, IDs like S001
Character	a single character	grade 'A', size 'M', Y/N code
Boolean	TRUE / FALSE (yes / no)	Paid?, Member?, In stock?
Integer	whole numbers	age, quantity, number of pages
Real	numbers with a decimal part	price 4.99, height 1.72
Date / time	dates and times	date of birth, appointment time

A **phone number** is text: it can start with 0, may contain spaces or +, and is never used in calculations.

Validation checks

Check	What it checks	Example
Range	value is between limits	Age 11 to 18
Length	number of characters	password ≥ 8; phone 11 digits
Type	right data type	Age must be an integer
Presence	not left blank	Surname must be entered
Format	matches a pattern	ID = S + 3 digits; date DD/MM/YYYY
Check digit	extra digit calculated from the others, to catch typing errors	ISBN, barcode

Defining a table from requirements

Give the table a name, list each field with a sensible **data type**, choose the **primary key**, and add **validation** where it helps. Example – a library stores each book's ID (e.g. B0042), title, price, date bought, whether it is on loan and its number of pages:

Field	Data type	Validation
BookID (primary key)	Text	format: B + 4 digits; presence
Title	Text	presence
Price	Real	range: 0 to 100
DateBought	Date/time	format DD/MM/YYYY
OnLoan	Boolean	type
Pages	Integer	range: 1 to 2000

SQL – Structured Query Language

Used to search (query) a database. The spec is limited to `SELECT, FROM, WHERE, ORDER BY … ASC/DESC, SUM, COUNT, AND, OR.`

```
SELECT Field1, Field2
FROM TableName
WHERE Condition
ORDER BY Field ASC;
```

- **SELECT** – which fields (columns) to show · **FROM** – which table · **WHERE** – which records (optional) · **ORDER BY** – how to sort them (optional).
- **Order** of clauses: `SELECT → FROM → WHERE → ORDER BY`.
- `SELECT *` shows **all** fields. Separate field names with **commas**.
- `ORDER BY Field ASC` = ascending (A→Z, smallest first); `DESC` = descending (Z→A, largest first). The spec calls these "ORDER BY ASCENDING / DESCENDING" – write `ASC` / `DESC`.
- `SUM(Field)` adds up a numeric field → **one value**. `COUNT(*)` counts matching records → **one value**.
- `AND` – both conditions true; `OR` – at least one true.
- Comparisons: `= <> > < >= <=` (`<>` means not equal).
- Text in quotes: `Form = '11A'`. Numbers without quotes: `Score > 50`. Boolean: `Paid = TRUE`.
- Field and table names must match the table **exactly**.

Worked examples – table STUDENT

StudentID	FirstName	Surname	Form	Age	Score	Paid
S001	Amy	Khan	11A	15	78	TRUE
S002	Ben	Lewis	11B	16	64	FALSE
S003	Cara	Jones	11A	16	91	TRUE
S004	Dev	Patel	11C	15	55	TRUE
S005	Ella	Smith	11A	15	83	FALSE
S006	Finn	Brown	11B	16	70	TRUE

```
SELECT FirstName, Surname
FROM STUDENT
WHERE Form = '11A'
ORDER BY Surname DESC;
```

Output:

FirstName	Surname
Ella	Smith
Amy	Khan
Cara	Jones

```
SELECT COUNT(*)
FROM STUDENT
WHERE Age = 16;
```

Output: `3`

```
SELECT SUM(Score)
FROM STUDENT
WHERE Form = '11B';
```

Output: `134`

```
SELECT FirstName
FROM STUDENT
WHERE Score > 75 AND Paid = TRUE;
```

Output:

FirstName
Amy
Cara

```
SELECT FirstName, Score
FROM STUDENT
WHERE Form = '11C' OR Score >= 80
ORDER BY Score ASC;
```

Output:

FirstName	Score
Dev	55
Ella	83
Cara	91

```
SELECT StudentID
FROM STUDENT
WHERE Age < 16
ORDER BY StudentID ASC;
```

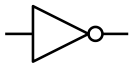
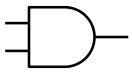
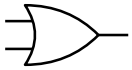
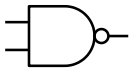
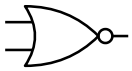
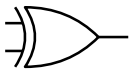
Output:

StudentID
S001
S004
S005

Identifying output: go through the table row by row and tick the rows that meet the WHERE condition; keep **only the fields in SELECT, in that order**; then sort if there is an ORDER BY. COUNT and SUM give a single number.

Common mistakes: missing commas between fields · text without quotes (or numbers in quotes) · WHERE after ORDER BY · mixing up COUNT (how many) and SUM (add up) · misspelt field or table names · forgetting DESC when the question says "highest first".

5 Boolean logic

Gate	Symbol	What it does	Expression	Truth table
NOT		Inverts the input. The **only gate with one input**.	X = NOT A	A X 0 1 1 0
AND		1 only when **both** inputs are 1.	X = A AND B	A B X 0 0 0 0 1 0 1 0 0 1 1 1
OR		1 when **at least one** input is 1.	X = A OR B	A B X 0 0 0 0 1 1 1 0 1 1 1 1
NAND		NOT AND – 0 only when both inputs are 1. AND symbol + bubble.	X = A NAND B	A B X 0 0 1 0 1 1 1 0 1 1 1 0
NOR		NOT OR – 1 only when both inputs are 0. OR symbol + bubble.	X = A NOR B	A B X 0 0 1 0 1 0 1 0 0 1 1 0
XOR		Exclusive OR (EOR) – 1 when the inputs are **different**. OR symbol + extra curved line.	X = A XOR B	A B X 0 0 0 0 1 1 1 0 1 1 1 0

Rules for the exam

- **NOT** is the only single-input gate; **all other gates have exactly two inputs**.
- Logic circuits have at most **three inputs and one output**.
- Draw circuits **exactly as the statement or expression is given – do not simplify**.
- A **bubble** (small circle) on a gate means NOT – the output is inverted (NOT, NAND, NOR).
- Use the standard symbols above, with the inputs on the left and the output on the right.

Truth tables

- 2 inputs → **4 rows**; 3 inputs → **8 rows** (2^n rows).
- Write the inputs counting up in binary: 000, 001, 010, 011, 100, 101, 110, 111 (C changes every row, B every 2, A every 4).
- For a long expression, add a **working column** for each part, then combine them.

Worked example: X = (A AND B) OR NOT C

A	B	C	A AND B	NOT C	X
0	0	0	0	1	1
0	0	1	0	0	0
0	1	0	0	1	1
0	1	1	0	0	0
1	0	0	0	1	1
1	0	1	0	0	0
1	1	0	1	1	1
1	1	1	1	0	1

Output column: **1, 0, 1, 0, 1, 0, 1, 1**

Problem statement → expression

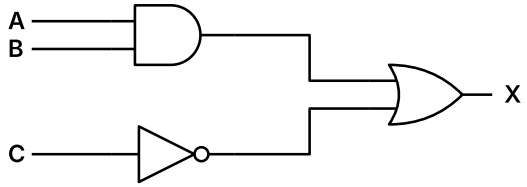
1. Identify the inputs (give each a letter) and the output.
2. Turn the words into operators: "and", "both" → AND; "or", "either" → OR; "not", "is off", "is not" → NOT.
3. Use brackets to show which parts go together.

- "An alarm X sounds if the door is open (A) **and** the system is armed (B), **or** if a window is broken (C)" → **X = (A AND B) OR C**
- "An alarm X sounds if it is armed (S) **and** either the door (D) **or** the window (W) is open" → **X = S AND (D OR W)**

- "A heater X is on if the temperature is low (T) **and** it is daytime (D), **or** the override (S) is on" → **X = (T AND D) OR S**
- "A light X is on when the switch (S) is on **and** it is **not** daylight (L)" → **X = S AND NOT L**

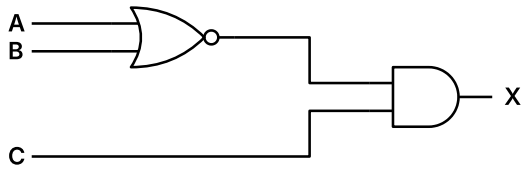
Expression → circuit

One gate for each operator. Start with the **innermost brackets**, then join their outputs with the next gate, ending with the gate that gives X. Example: **X = (A AND B) OR NOT C**



Circuit → expression

Start at the inputs and work towards the output, writing each gate's output as a bracketed expression.



NOR gate: **(A NOR B)**; then AND with C → **X = (A NOR B) AND C** (also written **NOT (A OR B) AND C**).

A	B	C	A NOR B	X
0	0	0	1	0
0	0	1	1	1
0	1	0	0	0
0	1	1	0	0
1	0	0	0	0
1	0	1	0	0
1	1	0	0	0
1	1	1	0	0

Truth table → expression

For each row where X = 1, AND the inputs together (use NOT for any input that is 0), then OR those terms.

A	B	C	X
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

X = 1 in rows 101 and 111 → **X = (A AND NOT B AND C) OR (A AND B AND C)**. Any correct equivalent is accepted (this one equals A AND C).

Two inputs, X = 1 when A = 0, B = 1 and when A = 1, B = 0 → X = (NOT A AND B) OR (A AND NOT B) – which is A XOR B.

More output columns to check yourself on

Expression	Output for rows 000 → 111
NOT (A OR B) AND C	0, 1, 0, 0, 0, 0, 0, 0
(A XOR B) AND C	0, 0, 0, 1, 0, 1, 0, 0
(A OR B) AND NOT C	0, 0, 1, 0, 1, 0, 1, 0
(A NAND B) OR C	1, 1, 1, 1, 1, 1, 0, 1
A AND (B OR C)	0, 0, 0, 0, 0, 1, 1, 1

Common mistakes: rows not in binary order · using a 3-input gate (only 2 inputs allowed) · simplifying the circuit · forgetting the bubble on NAND / NOR · mixing up XOR (different) with OR (at least one).

6 Checklist – can you…?

☐ explain what procedures, functions and parameters are, and the difference between a procedure and a function
☐ write procedures and functions with and without parameters (up to three), and call them correctly
☐ explain local and global variables, their scope, and why local variables are preferred
☐ use MOD, DIV, ROUND and RANDOM, and explain the benefits of library routines
☐ explain why programs store data in files
☐ open, read, write and close files – single items and lines of text
☐ explain how to make a program maintainable: meaningful identifiers (variables, constants, arrays, procedures, functions), comments, procedures and functions
☐ define a single-table database: fields, records, validation
☐ suggest data types: text/alphanumeric, character, Boolean, integer, real, date/time
☐ explain the purpose of a primary key and pick a suitable one
☐ read, complete and write SQL with SELECT, FROM, WHERE, ORDER BY ASC/DESC, SUM, COUNT, AND, OR
☐ work out the output of an SQL query on a given table
☐ draw and recognise the NOT, AND, OR, NAND, NOR and XOR symbols and know their truth tables
☐ draw a circuit from a problem statement, expression or truth table (no simplifying)
☐ complete a truth table from a problem statement, expression or circuit
☐ write an expression from a problem statement, circuit or truth table

7 Quick test (answers below)

1. Give two differences between a procedure and a function.
2. Write the header of a function `Area` that takes a length and a width (both real) and returns a real.
3. What are `MOD(29, 6)` and `DIV(29, 6)`?
4. Why should a program use local variables rather than global ones?
5. What happens to the contents of an existing file opened FOR WRITE?
6. Write pseudocode to store the value of `Score` in Score.txt.
7. State three ways of making a program maintainable.
8. Suggest a data type for: a price; a member's date of birth; whether a fee is paid; a phone number.
9. Why is Surname a poor primary key?
10. Write SQL to show the FirstName of every student in 11B, highest Score first (table STUDENT).
11. What is the output of `SELECT COUNT(*) FROM STUDENT WHERE Paid = FALSE;`?
12. Which gate outputs 1 only when both inputs are 0?
13. How many rows does a 3-input truth table have?
14. Write the expression: "A pump X runs if the tank is low (L) and the valve is not closed (V)".

Answers

1. A function returns a value, a procedure doesn't; a function is used in an expression, a procedure is run with CALL; a function header ends RETURNS type.
2. `FUNCTION Area(Length : REAL, Width : REAL) RETURNS REAL`
3. `MOD(29, 6) = 5`, `DIV(29, 6) = 4`
4. They can't be changed by mistake elsewhere; names can be reused; memory is freed; subroutines are self-contained and easier to test.
5. They are overwritten – the file starts empty.
6. `OPENFILE "Score.txt" FOR WRITE` · `WRITEFILE "Score.txt", Score` · `CLOSEFILE "Score.txt"`
7. Meaningful identifiers; comments; procedures and functions (also indentation, constants).
8. Real; date/time; Boolean; text.
9. Two people can share a surname, so it isn't unique.
10. `SELECT FirstName FROM STUDENT WHERE Form = '11B' ORDER BY Score DESC;`
11. 2
12. NOR
13. 8
14. **X = L AND NOT V**